# Dessiner un graphe directement en espace écran

Notre proposition d'orientation et nos questions

---

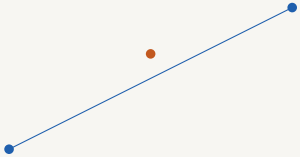
Melissa Colin  Quentin Lopez  Yani

Octobre 2026

ENSEIRB-MATMECA 3A, projet semestriel · Encadrant : Romain Giot (LaBRI)

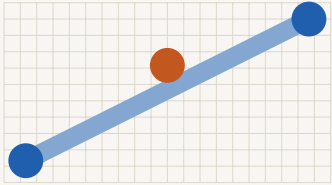
# Le problème : optimal en géométrie, illisible à l'écran

**Espace géométrique**



Le nœud orange ne touche pas le segment : le coût géométrique est nul.

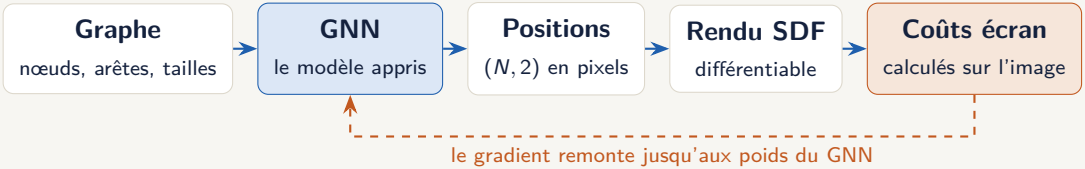
**Espace écran (pixels)**



Une fois rendue, l'arête épaisse recouvre le nœud orange.

Les méthodes classiques ($(GD)^2$, SGD, FORBID) optimisent des points et des segments sans épaisseur. Le sujet propose de calculer les coûts sur l'image rendue.

# Notre idée : un réseau qui place les nœuds directement en pixels

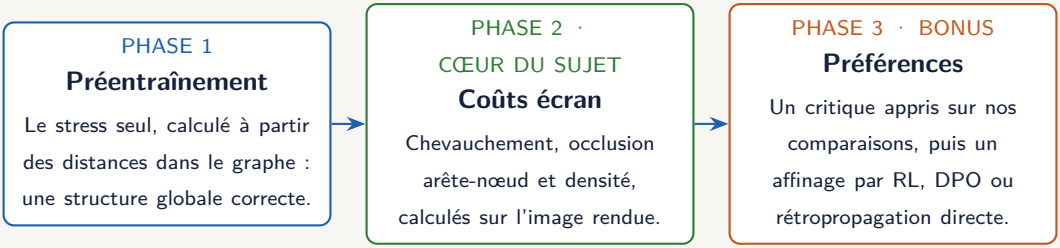



**Entraîné une fois**
Un nouveau graphe se dessine en un seul passage, sans optimisation propre à chaque graphe.

**Sortie continue**
Des coordonnées réelles dans l'image, pour que le gradient passe.

**Taille variable**
Un GNN accepte des graphes de 13 à 100 nœuds, comme ceux de Rome.

# Entraînement : sans dessins de référence, en trois phases




**Pourquoi commencer par le stress.** Loin du bord d'une forme, la sigmoïde des SDF est plate : un réseau qui place ses nœuds au hasard ne recevrait presque aucun gradient des coûts écran. DeepGD entraîne déjà un GNN de cette façon, sans supervision.

# Une piste à tester : rendre le modèle attentif aux arêtes

Les arêtes ne sont pas prédites : ce sont des segments entre les nœuds. Pourtant, la plupart des défauts à l'écran viennent d'elles.

**A. Tête auxiliaire**
Une tête en plus prédit les défauts de chaque arête (croisements, nœud caché, longueur). Entraînée par une perte auxiliaire, elle oblige le GNN à « comprendre » les arêtes.

**B. Raffinement itératif**
Une première passe propose des positions. On calcule exactement les défauts de chaque arête, puis une seconde passe les reçoit dans son attention et corrige.

**Hypothèse**
On compare le modèle de base, A et B sur les mêmes graphes (ablation).

# Points de vigilance : les risques et nos parades

| Risque | Parade |
|---|---|
| Deux nœuds sur des pixels voisins se recouvrent quand même | Contrainte réelle : distance des centres $\geq r_i + r_j$ |
| Choisir un pixel (arg max) bloque le gradient | Sortie en coordonnées continues |
| Mémoire de l'ordre de $(N + E) \times H \times W$ | Petits graphes (Rome : 13 à 100 nœuds), carte somme $S = \sum_i I_i$ |
| Gradients des SDF presque nuls loin des formes | Préentraînement par le stress |
| Préférences : beaucoup d'annotations | Phase 3 en bonus, DPO ou critique préentraîné |

## Un modèle évalué avec ce qu'il minimise gagne forcément.

Loi de Goodhart : quand une mesure devient un objectif, elle cesse d'être une bonne mesure.

### Des métriques hors de la loss

Par exemple les croisements, si on ne les optimise pas, et des métriques de lisibilité de la littérature.

### Des méthodes de référence

Kamada-Kawai, SGD, FORBID sur les mêmes graphes, toutes mesurées sur le rendu final.

# Organisation : trois rôles, un contrat commun

### A. Modèle (Melissa)

GNN, préentraînement par le stress, inférence

### B. Rendu et données

Rendu SDF différentiable, chargement de Rome, boucle d'entraînement

### C. Loss et évaluation

Métriques, loss, comparaison aux méthodes de référence

LE CONTRAT, DÉJÀ DANS LE CODE

```
GraphInput
  edge_index (2, E)
  node_size  (N,)
  dist       (N, N)

Layout
  positions (N, 2)
  width, height
```

06/11 · Jalon 1 | 18/12 · Jalon 2 | 27/01 · Soutenance

# Nos questions (1/2) : orientation et évaluation

1. Un modèle appris, sans supervision, qui dessine en coordonnées écran : cette orientation vous convient-elle, ou faut-il commencer par l'optimisation directe d'un graphe ?
2. Quelles entrées viser au départ : la structure et la taille des nœuds, ou aussi épaisseurs, couleurs et formes ?
3. Quelle taille de graphes, quelle résolution d'image et quel temps de calcul vous paraissent raisonnables ?
4. Quelles métriques et quel protocole pour comparer un dessin optimisé en espace écran à un dessin géométrique ?
5. Quelles méthodes de référence comparer en priorité, et avec quelles implémentations ?

# Nos questions (2/2) : organisation et rendus

6. Qu'attendez-vous aux jalons du 06/11 et du 18/12 ? Un état de l'art écrit, ou seulement à l'oral ?
7. Quels livrables finaux : le code seul, un notebook de démonstration, un rapport ?
8. Le sujet mentionne des exemples de code : comment y accéder ?
9. Avons-nous accès à des GPU (LaBRI ou école), ou travaillons-nous sur nos machines ?
10. Le sujet cite GitLab : est-il obligatoire, ou un dépôt GitHub privé avec accès pour vous convient-il ?